

Ch2_Classical Encryptions

Classical Ciphering

Introduction

1. Caesar cipher 凯撒密码

1.1. Cryptanalysis – Brute Force 暴力破解

1.1.1. This works because:

1.1.2. Why key length determines feasibility?

1.1.3. 为什么 暴力破解可行?

2. Monoalphabetic Cipher 单表替换密码

2.1. 密钥空间 (Keyspace = 26!)

2.2. 为什么密钥空间这么大仍然不安全?

2.2.1. 分析方式: Statistical Attack

3. Vigenère cipher 维吉尼亚密码

3.1. 密钥空间

4. Playfair cipher 普莱费尔密码

4.1. Playfair 加密规则

Symmetric Cipher Model 对称加密

1. five ingredients

2. Symmetric Cipher Model 对称加密模型

2.1. Traditional encryption algorithms are private-key

3. Unconditional secure VS Computational secure

【拓展】为什么现代密码都是 Computational secure?

Classical Ciphering

Introduction

古典密码是现代密码学出现之前，人类用于保护信息的主要方式。它们通常基于 **字母替换 (substitution)** 或 **字母重排 (transposition)**

1. **Substitution Ciphers** 替换密码，用另一个字母替换原字母
2. **Transposition Ciphers** 重排密码不改变字母本身，而是改变字母的顺序
3. **Product Cipher** Combines multiple substitution and transposition ciphers to increase security. 这一思想直接启发了现代分组密码（如 AES）

1. Caesar cipher 凯撒密码

■ Replace each letter by a shift operation from 1 to 25

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

■ Mathematically expressed as the function:

$$C = E(k, p) = (p + k) \bmod 26 \quad \text{加密: Ciphertext} = (\text{plaintext number} + \text{key}) \bmod 26$$

$$p = D(k, C) = (C - k) \bmod 26 \quad \text{解密: Plaintext} = (\text{ciphertext number} - \text{key}) \bmod 26$$

1. 把明文中的每个字母按一个固定的偏移量（密钥）进行移动，偏移范围为 1 到 25
2. 为了用数学表达凯撒密码，需要把字母映射为数字
 - 归根到底，体现在字母本身：加密就是对应**字母表向前移动 k 位**

plain: meet me after the toga party
 cipher: PHHW PH DIWHU WKH WRJD SDUWB

1.1. Cryptanalysis – Brute Force 暴力破解

KEY	PHHW PH DIWHU WKH WRJD SDUWB
1	oggv og chvgt vjg vqic rctva
2	nifu nf bgufs uif uphb qbsuz
3	meet me after the toga party
4	ldds ld zesdq sgd snfz ozqsx
5	kccr kc ydrpc rfc rmey nyprw
6	jbbq jb xcqbo qeb qldx mxoqv
7	iaap ia wbpap pda pkcw lwnpu
8	hzzo hz vaozm ocz ojbv kvmot
9	gyyn gy uznyl nby niau julns
10	fxxm fx tymxk max mhzt itkmr

A brute-force attack (also called **exhaustive key search**) systematically tests **all possible keys or passwords** until one produces a valid plaintext.

1.1.1. This works because:

- Every encryption algorithm has a **finite keyspace**.
- If the keyspace is small enough, an attacker can enumerate 枚举 all keys.
- The correct key is found when the decrypted output becomes meaningful.

1.1.2. Why key length determines feasibility?

For a key of **n bits**, the total keyspace is: 2^n

On average, a brute-force search needs half the keyspace: 2^{n-1}

This exponential growth is why modern cryptosystems use large keys.

Key length	Keyspace	Approx. brute-force time
40 bits	$2^{40} \approx 1.1$ trillion	< 2 minutes
56 bits (DES)	2^{56}	~83 days
64 bits	2^{64}	~58 years
128 bits (AES-128)	2^{128}	Longer than the age of the universe

256 bits (AES-256)	2256	Physically impossible
--------------------	------	-----------------------

Key Size (bits)	No. of keys	Time (1 encryption/ μ s)	Time (10^6 encryptions/ μ s)
32	$2^{32} = 4.3 \times 10^9$	35.8 minutes	2.15 millisec
56 (DES)	$2^{56} = 7.2 \times 10^{16}$	1142 years	10.01 hours
128 (AES)	$2^{128} = 3.4 \times 10^{38}$	5.4×10^{24} years	5.4×10^{18} years
168 (3DES)	$2^{168} = 3.7 \times 10^{50}$	5.9×10^{36} years	5.9×10^{30} years
26 characters permutation	$26! = 4 \times 10^{26}$	6.4×10^{12} years	6.4×10^6 years



Classical cipher 的 key 是 **人类可读的结构**，不是二进制位串，因此 classical cipher 的 keyspace 很小，容易 brute force；

Modern cipher 的 key 是 binary，keyspace = 2^n (指数级) → brute force 不可能

This is why brute-force cryptanalysis **fails completely** against modern symmetric ciphers

1.1.3. 为什么 暴力破解可行？

- Each alphabet is mapped to another alphabet by shifting each to the left or right **circularly**.
- Could simply try each in turn by an exhaustive key search.
- Encryption and decryption algorithm are **known**
- **Too small** for brute-force.
- Language of plaintext is known and **easily** recognizable.
- If language of plaintext is unknown, the plaintext output may not be recognizable.

2. Monoalphabetic Cipher 单表替换密码

Shuffle the letters arbitrarily 把字母表 任意打乱，并建立一个 一对一映射



2.1. 密钥空间 (Keyspace = 26!)

- 凯撒密码：只有 25 个密钥
- 单表替换密码：**26! (阶乘) 个密钥** $26! = 403,291,461,126,605,635,584,000,000$

2.2. 为什么密钥空间这么大仍然不安全？

因为 **字母频率不会改变**。

例如英文中：

- E 最常出现
- T、A、O 次之
- Q、Z 很少出现

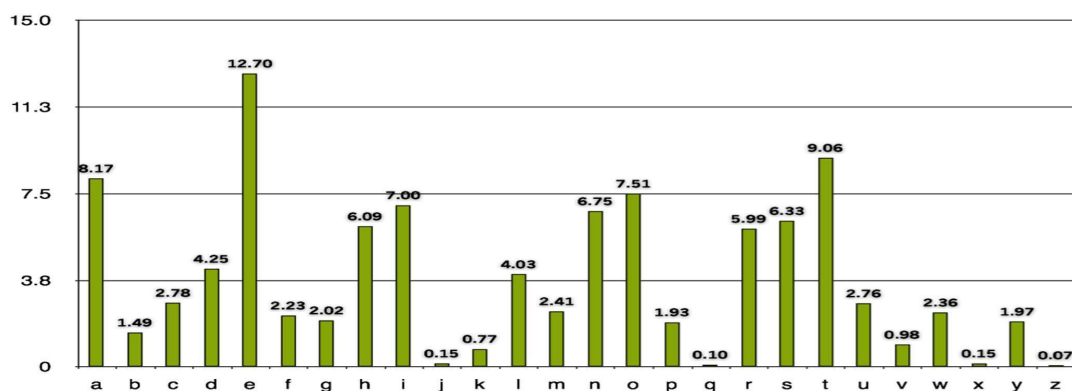
攻击者可以通过 **频率分析 (frequency analysis)** 轻松破解。

所以，**密钥空间大 ≠ 安全；结构弱才是致命问题**。

2.2.1. 分析方式：Statistical Attack

Monoalphabetic Cipher 虽然把字母替换了，但 **不会改变字母出现的频率分布**。

- 明文中出现最多的字母（例如英文的 e）
- 在密文中仍然会出现最多 -> Monoalphabetic Cipher 是 **一对一替换**，所以密文的频率分布和明文的频率分布形状一致
- 只是换成了另一个字母（例如 P）



攻击步骤：

- ① 计算密文中每个字母的出现频率
- ② 对照英文频率表
- ③ 替换密文中的这些字母

■ Ranking of frequented letters

Letter	(%)
P	13.3
Z	11.7
S	8.3
U	8.3
O	7.5
M	6.7



Letter	(%)
e	12.7
t	9.1
a	8.2
o	7.5
i	7.0
s	6.3

■ Ciphered Text

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
ta e e te a that e e a a
VUEPHZHMDSHZOWSFPAPDTSVPQZWMXUZHXS
et ta tha e e e ae th t a
EPYEPDPDZSZUPOMBZWPFPZHMJDUTMOHMQ
e e e tat e the t

3. Vigenère cipher 维吉尼亚密码

维吉尼亚密码是一个 **多重凯撒密码**。明文的每个字母根据关键字中的不同字母进行不同的偏移

■ Vigenère cipher is a multiple Caesar cipher

$$K = \{K_1, K_2, \dots, K_n\}, n \leq m,$$

- 密钥是一个长度为 n 的字母序列
- 明文长度为 m
- 密钥重复使用直到覆盖整个明文

■ Repeat from start after n letters in message

$$P = \{P_1, P_2, \dots, P_m\}.$$

■ Mathematically:

$$C = E(K, P) = (P + K) \bmod 26$$

$$P = D(K, C) = (C - K) \bmod 26$$

例,

- 明文: ATTACKATDAWN
- 密钥: LEMON
- 重复密钥: LEMONLEMONLE
- 用公式 $C = (P + K) \bmod 26$
- 得到密文: **LXFOPVEFSNHR**

3.1. 密钥空间

在密钥长度确定的情况下:

- Vigenère 的密钥空间取决于密钥长度 n , 大小为: 26^n
- 因为密钥是由 n 个字母组成, 每个字母有 26 种选择。

4. Playfair cipher 普莱费尔密码

Playfair is the best-known digram (two-letter) substitution cipher. It encrypts plaintext **two letters at a time**, using a **5×5 matrix** built from a keyword. Playfair 是最著名的 **双字母替换密码**。它每次加密 **两个字母**, 使用一个由关键字构造的 **5×5 字母矩阵**。

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

关键字: **MONARCHY**

构造规则:

1. 去重: M O N A R C H Y
2. 填入矩阵
3. 剩余字母按顺序补齐 (I/J 合并)

4.1. Playfair 加密规则

① Rule 1: 重复字母 → 插入 X

Repeating plaintext letters that are in the **same pair** are separated with a filler letter (X).

注意: 如果不在同一个 pair 中, 那么不需要 fill X

② Rule 2: 同一行 (Row)

如果两个字母在同一行 → 各自替换为**右边**的字母 (循环) :

③ Rule 3: 同一列 (Column)

如果两个字母在同一列 → 各自替换为**下面**的字母 (循环) :

④ Rule 4: 矩形规则 (Rectangle Rule)

既不同行也不同列 → 形成一个矩形: 每个字母替换为 **同一行、对方的列** 的字母。

解密, 反之。

■ Plain text: "Nice to meet you" → NI CE TO ME ET YO UX

■ Key: Monarchy

■ Ciphred text: _____

	M	O	N	A	R
	C	H	Y	B	D
同行替右 同列替下 右下加密 反之解密	E	F	G	I/J	K
	L	P	Q	S	T
	U	V	W	X	Z

Symmetric Cipher Model 对称加密

1. five ingredients

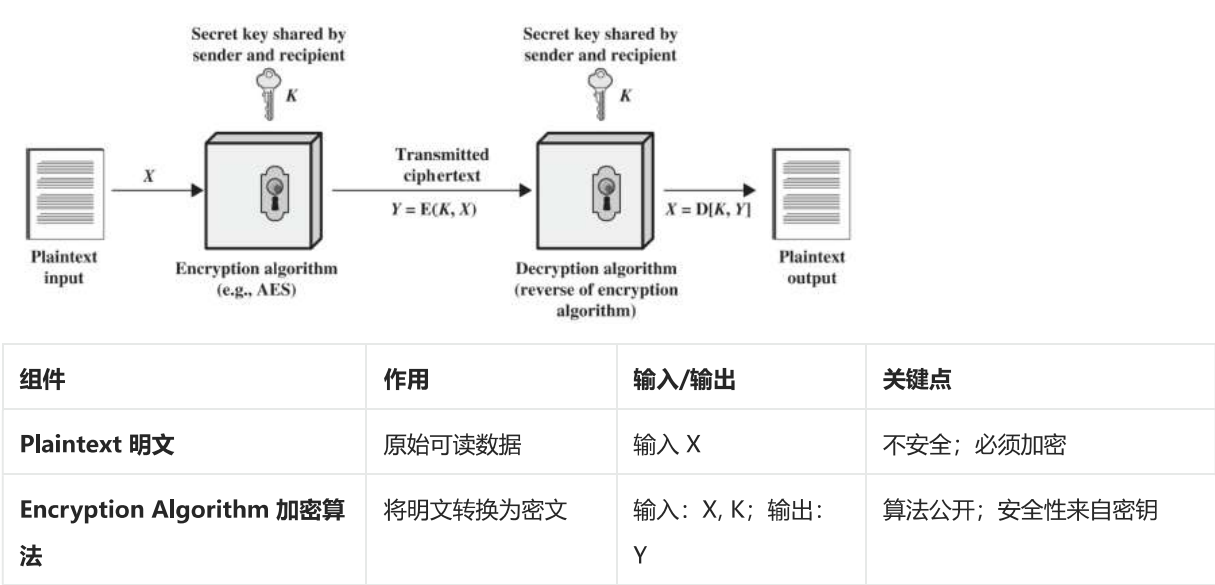
- **Plaintext:** This is the **original** intelligible message or data that is fed into the algorithm as **input**.
- **Encryption algorithm:** The encryption algorithm performs various substitutions and transformations on the plaintext.
- **Secret key:** The secret key is also **input to the encryption algorithm**. The key is a value independent of the plaintext and of the algorithm. The algorithm will produce a different output depending on the specific key being used at the time. The exact substitutions and transformations performed by the algorithm depend on the key.

- Ciphertext:
 - This is the scrambled message produced as output. It depends on the plaintext and the secret key. For a given message, two different keys will produce two different ciphertexts. The ciphertext is an apparently random stream of data and, as it stands, is unintelligible.
- Decryption algorithm:
 - This is essentially the encryption algorithm run in reverse. It takes the ciphertext and the secret key and produces the original plaintext.

组成部分	定义	作用	关键特性
Plaintext 明文	原始可读数据；未加密内容	输入到加密算法中	不包含密钥；安全性完全依赖算法与密钥
Encryption Algorithm 加密算法	对明文进行替换、置换、混淆的数学函数	根据密钥执行具体加密步骤	算法公开；安全性来自密钥而非算法隐藏
Secret Key 密钥	控制算法行为的二进制值	决定加密输出；同算法不同密钥 → 不同密文	必须保密；独立于明文与算法；决定安全性
Ciphertext 密文	加密后的随机化输出	用于安全传输或存储	看起来随机；无密钥无法理解；
Decryption Algorithm 解密算法	加密算法的逆过程	用密钥将密文还原成明文	必须与加密算法严格匹配；同密钥才能解密

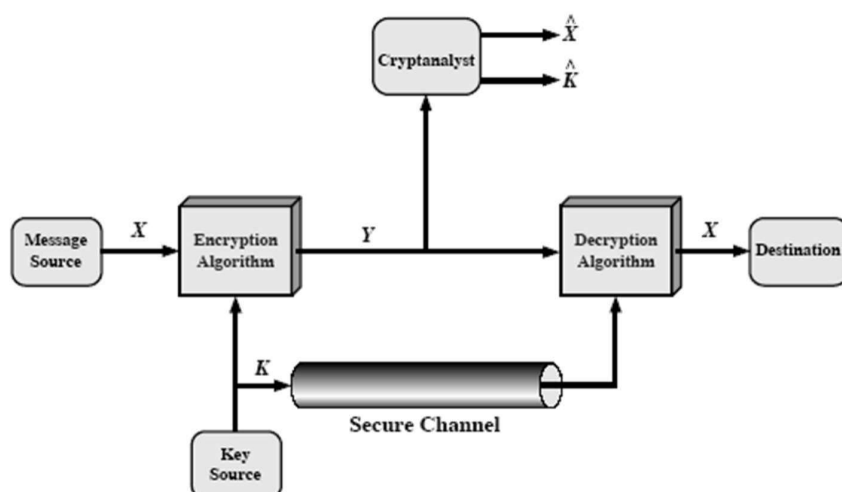
2. Symmetric Cipher Model 对称加密模型

发送方和接收方使用同一个密钥 K 来加密与解密



Secret Key 密钥	控制加密与解密行为	输入到加密与解密算法	必须共享；必须保密；必须一致
Ciphertext 密文	加密后的随机化输出	输出 Y	无密钥无法理解；可安全传输
Decryption Algorithm 解密算法	将密文还原为明文	输入：Y, K；输出：X	必须使用同一个密钥 K

2.1. Traditional encryption algorithms are **private-key**



- 发送方和接收方必须共享同一个密钥 K
- 密钥必须通过 **安全信道 (Secure Channel)** 传输
- 如果密钥泄露 → 加密系统完全失效
- 密钥管理是对称加密中最难的部分

● Key

■ The parameter which selects which individual transformation is used, and is selected from a key space K.

1. 密钥不是一个“值”，而是一个 **选择器 (selector)**。
2. 加密算法不是一个函数

加密算法 E 是一个 **函数族 (family of functions)** : $\{E(K, \cdot) \mid K \in \text{Keyspace}\}$

每一个密钥 K 都对应一个不同的函数。

例如 AES-128:

- 密钥空间 = 2^{128}
- 也就是说 AES 有 2^{128} 个不同的函数
- 密钥 K 决定你选的是哪一个

如果加密是一个单独的函数: $Y=E(X)$

那就意味着：所有明文 X 都用同一个变换，所有密文 Y 都是固定的，没有密钥，所有人都能解密

■ More formally we can define the cryptographic system as a single parameter family of invertible transformations.

- 每个 K 对应一个不同的可逆变换
- **可逆 (invertible)** 意味着 $D(K, E(K, X)) = X$
如果不可逆 → 无法解密 → 加密系统不可用。

With the message X and the encryption key K as input, the encryption algorithm forms the ciphertext $Y = [Y_1, Y_2, \dots, Y_N]$. We can write this as

$$Y = E(K, X) \quad \text{加密}$$

This notation indicates that Y is produced by using encryption algorithm E as a function of the plaintext X , with the specific function determined by the value of the key K .

The intended receiver, in possession of the key, is able to invert the transformation:

$$X = D(K, Y) \quad \text{解密}$$

3. Unconditional secure VS Computational secure

- **Unconditional secure (无条件安全)** : 不管攻击者有多少计算能力, 都**无法**从密文推断出明文。
 - One-Time Pad (一次一密) : 密文 C 对攻击者来说完全随机 -> 信息不足
 - 密钥长度 = 明文长度
 - 密钥完全随机
 - 密钥只使用一次
 - 密钥必须安全分发
- **Computational secure (计算安全)** : 理论上能破解, 但需要的计算量大到现实中不可能做到。
 - AES-128 brute-force → 2^{128} 次尝试
 - AES-256 brute-force → 2^{256} 次尝试
 - RSA → 需要分解大整数
 - ECC → 需要解决离散对数问题

这些问题在数学上 **不是不可能**, 只是 **太难**。

● Unconditional secure

- No matter how much computer power is available, the cipher cannot be broken since the ciphertext provides insufficient information to uniquely determine the corresponding plaintext.

● Computational secure

- Given limited computing resources (e.g. time needed for calculations is greater than age of universe), the cipher cannot be broken
- Cost of breaking exceeds the value of information
- Time required exceeds the lifetime of information

【拓展】为什么现代密码都是 Computational secure?

因为:

- 无条件安全 (One-Time Pad) 需要密钥长度 = 明文长度
- 密钥必须完全随机

- 密钥必须安全分发
- 密钥只能使用一次

这在现实系统中几乎不可能做到。

所以现代密码学采用：

- **计算安全 (Computational secure)**
- 依赖数学难题 (复杂度理论)
- 依赖足够大的密钥空间 (2^n)